QUESTION For Perplexity.ai >
‘tell me about SARIMA modeling and if that might apply to analysis of the uploaded dataset. The dataset is a matrix distribution of the number of parts used on a specific day where there are a total of 39 parts (1-39). the dataset headers and first row >>
date,P_1,P_2,P_3,P_4,P_5,P_6,P_7,P_8,P_9,P_10,P_11,P_12,P_13,P_14,P_15,P_16,P_17,P_18,P_19,P_20,P_21,P_22,P_23,P_24,P_25,P_26,P_27,P_28,P_29,P_30,P_31,P_32,P_33,P_34,P_35,P_36,P_37,P_38,P_39,
9/8/2008,0,0,1,0,0,1,2,2,0,0,1,1,2,0,0,2,1,1,0,2,1,1,2,0,1,1,1,1,2,1,0,0,1,0,1,0,0,0,1,30 P_1 to P_39 represent the various part numbers that might be used. A '0' in a cell indicates that none of that part number where required that day. For example for 9/8/2008, Col P_1 = '0' so no parts were required, but col P_3 = '1' means 1 part number P_3 (part number 3) was required/used. Col (P_7) = '2' which means 2 of part number 7 (P_7) where required. What I want to do is use the temporal uploaded dataset to model then predict the 20 most likely part numbers 1-39 that will be required for the next day/event. SARIMA modeling is probably not the only methodology that we can use as a part of this pipeline. I would like your detailed assessment and detailed technical concept paper that I can present to a Bmad-method TEAM ref > (https://github.com/bmad-code-org/BMAD-METHOD) to then develop a prd.md and architecture.md along w/ a Claude Code Dev Ops team in my Windsurf IDE to develop the project fully’

c5_aggregated_matrix.csv > uploaded

Completed 2 steps

SARIMA can be a useful component for your problem, but it should be applied per-part (univariate series) and embedded in a broader probabilistic ranking pipeline that handles 39 correlated count series and “top-K next-day” prediction.

Below is a concept paper–style outline you could hand to a BMAD-METHOD team and a Claude Code DevOps team.

Problem framing

You have a daily time series matrix: one row per **date**, columns **P_1…P_39** with non-negative integer counts of units used that day, plus a final column that is always 30 in the sample (likely a total or constant).

Your core tasks:

- Next-day prediction: for a given day t, estimate the distribution of counts $Y_{t+1}^{(j)}$ for each part $j \in \{1, \dots, 39\}$.
- Ranking: from these predictions, output the 20 most likely parts "required" for $t+1$, with some score (e.g., expected count, probability of being non-zero).

Key constraints:

- Data are discrete counts with many zeros and small positive integers (0–5 typical).
- There is likely strong common seasonality (day-of-week, month, etc.) and cross-part correlation (certain parts tend to be used together).

Where SARIMA fits

What SARIMA is

Seasonal ARIMA (SARIMA) is a univariate time-series model for (usually) continuous or approximately Gaussian data, modeling trend, autoregression, differencing, moving averages, and seasonal components.

For each part j, you can define a separate SARIMA model on the daily count series $\left\{Y_t^{(j)}\right\}$, potentially after transformation (e.g., log(1+count)) to make the distribution more Gaussian-like.

Strengths vs your data

SARIMA is potentially useful for:

- Capturing temporal autocorrelation: parts that are used today may be more or less likely tomorrow; SARIMA can encode that with AR and MA terms.
- Modeling seasonality: weekly or monthly patterns in counts can be captured with seasonal orders $(P, D, Q)_s$ (e.g., $s = 7$ for day-of-week).
- Producing forecasts and forecast intervals per part, which can be turned into expected counts or probabilities that count > 0.

Challenges:

- Count data and zero inflation violate SARIMA's normal-error assumptions; this can bias uncertainty estimates and occasionally the mean.

- You have 39 series; per-series SARIMA order selection and re-fitting can become operationally heavy if you want frequent re-training.
- SARIMA is inherently univariate; it does not directly use correlations between parts (e.g., assemblies, kits).

Conclusion: SARIMA is reasonable as a baseline per-part forecaster (after transformation), but it should sit alongside count-specific and multivariate methods.

Recommended modeling pipeline

1. Data preprocessing layer

Goals: clean, align, and feature-engineer the daily matrix.

- Convert date to a proper datetime index and ensure regular daily frequency (forward/backfill missing days with zeros).
- Split features:
    - Target matrix $Y_{t,j}$ = count of part j on day t.
    - Optional covariates X_t: day-of-week, month, holidays, promotions, production schedule, etc. (to be extended later).
- Identify and drop/rename the constant "Unnamed: 40" column or treat it as total usage if that's the correct semantic.
- Train/validation/test split: contiguous splits (e.g., first 70% of days for training, next 15% validation, last 15% test).

Deliverable: data_prep.py and a documented schema (for PRD/architecture) describing raw and processed tables.

---

2. Core forecasting layer (per part)

2.1 SARIMA baseline per part

For each part j:

- Transform $Y_t^{(j)}$ via $Z_t^{(j)} = \log(1 + Y_t^{(j)})$ or square root, to reduce skew and make residuals more Gaussian.
- Fit SARIMA or SARIMAX with exogenous variables (day-of-week dummies, etc.):

    - Model specification: SARIMAX$(p, d, q) \times (P, D, Q)_s$ with $s = 7$ or other period after exploratory autocorrelation/partial autocorrelation analysis.
    - Order selection: use information criteria (AIC/BIC) on a grid or auto-ARIMA, constrained to low orders for robustness.
- Forecast horizon: next 1–N days; your immediate need is $N = 1$.
- Back-transform to counts: $\hat{Y}_{t+1}^{(j)} = \max\,(0, \exp\,(\hat{Z}_{t+1}^{(j)}) - 1)$ and round or keep as real-valued expectation.

These forecasts provide a per-part **expected usage** for the next day and a variance (or prediction interval).

2.2 Count-time-series alternatives

Because your targets are small counts, include at least one count-specific baseline:

- Per-part Poisson or negative binomial regression with lagged counts and calendar effects as predictors (GLM with log link).
- State-space count models (e.g., Poisson dynamic linear models) if you want more Bayesian structure, though these are heavier to implement.

Hybrid approach: use SARIMA or SARIMAX for capturing autocorrelation and seasonality on a continuous scale, then calibrate the output to a discrete distribution (e.g., map mean and variance to a negative binomial per part).

Deliverable: models/sarima_baseline.py and models/poisson_nb_baseline.py with common interfaces.

---

3. Ranking and decision layer (top-20 parts)

Given, for each part j, a next-day predictive distribution or at least a mean and variance:

- Compute a score for each part:
    - Expected count: $s_j = \mathbb{E}[Y_{t+1}^{(j)}]$.
    - Or probability of at least one unit: $s_j = \Pr\,(Y_{t+1}^{(j)} \geq 1)$, derived from the Poisson/NB assumption with mean from SARIMA/GLM.
- Rank parts by score descending; choose the top 20.

- Optionally impose business constraints, e.g., minimum threshold on probability of use or expected count.

If you forecast a distribution (e.g., negative binomial), you can present richer metrics: expected usage, 90% upper bound, and probability of stockout given buffer levels.

Deliverable: ranking/top_k_selector.py plus a spec for ranking metrics and thresholds.

---

4. Multivariate and ML extensions

SARIMA is univariate; to exploit interactions between parts and richer features, plan for additional models:

4.1 Multivariate time series

- Vector autoregression (VAR/VARX): model all 39 parts jointly as a vector process, optionally with exogenous variables (production schedule).
- Dynamic factor models: extract latent factors that explain common co-movement (e.g., "assembly A production days") and use them in per-part models.

Given 39 series, a low-dimensional latent factor model feeding per-part regressions can provide better generalization than 39 independent SARIMAs.

4.2 Machine-learning forecasters

- Gradient boosted trees (XGBoost/LightGBM/CatBoost) per part or multi-output: features include past lags, moving averages, day-of-week, month, total usage, etc.
- Sequence models:
    - Temporal convolutional networks.
    - Encoder-decoder or transformer-based multivariate sequence models (e.g., TFT, DeepAR-style) designed for probabilistic forecasting of many related series.

Approach:

- First iteration: SARIMA + simple GLM baselines to establish a performance floor.
- Second iteration: tree-based and deep time-series models with direct multi-step forecasting and probabilistic outputs.

Deliverables: models/ml_forecasters.py, model registry, and clear evaluation comparisons.

---

5. Evaluation and metrics

You need metrics both for per-part forecast accuracy and top-K selection quality.

Per-part forecast metrics:

- Point forecasts: MAE, RMSE, MAPE (careful with zeros) per part and aggregated.
- Probabilistic: pinball loss for quantiles, CRPS if using full predictive distributions.

Top-20 ranking metrics:

- Precision@20: fraction of predicted top-20 that actually had $Y_{t+1}^{(j)} > 0$.
- Recall@20: fraction of actually used parts (with $Y_{t+1}^{(j)} > 0$) that appear in predicted top-20.
- Normalized discounted cumulative gain (nDCG) using actual counts as relevance.

Design evaluation as rolling-origin backtesting: repeatedly train on data up to time T, predict $T + 1$, slide forward, and aggregate metrics.

Deliverables: evaluation/backtesting.py and an evaluation section in prd.md.

---

6. System architecture concept

High-level components for architecture.md:

1. **Ingestion & storage**
    - Batch job to pull daily usage matrix into a warehouse (e.g., S3 + DuckDB/Parquet, or Postgres).
    - Versioned raw and cleaned tables.
2. **Feature & model service**
    - Offline pipeline (orchestrated via Airflow/Prefect/GitHub Actions) that runs daily:
        - Read last N years of daily usage.
        - Run data prep and feature engineering.
        - Fit or update models (SARIMA, GLM, ML models).

        - Store model artifacts (coefficients, scalers, hyperparameters) in object storage.

3. **Prediction service**
    - Scheduled job to generate next-day forecasts for all parts.
    - Ranking module to select top-20 parts and emit JSON or CSV (part_id, expected_count, probability_nonzero, rank).
4. **Monitoring & governance**
    - Track drift: rolling forecast error and ranking metrics; alert when performance degrades.
    - Log model versions, configuration, and data snapshots for reproducibility.

Integration with Windsurf + Claude Code:

- Infra as code and CI/CD: Dockerized modeling service, tests for ETL and forecasting, automatic deployment on main branch.
- Prompt-driven scaffolding of new models and experiments; code-generated PRs reviewed by human + BMAD-METHOD guidelines.

---

7. SARIMA-specific technical considerations

If you proceed with SARIMA as a first-class method, recommend:

- Global constraints on orders, e.g., $p, q, P, Q \in \{0,1,2\}$, $d, D \in \{0,1\}$, to avoid overfitting and reduce compute.
- Shared seasonal period for all parts (likely 7 days; test 5/7/30 depending on operational cycles).
- Diagnostic checks per part: residual autocorrelation, Ljung-Box test, residual distribution; flag series where SARIMA is clearly mis-specified and fall back to simpler models.
- Model selection strategy:
    - Either per-part auto-selection (auto-ARIMA) with guardrails.
    - Or small grid search on a handful of archetype parts, then reuse orders across similar parts.

You can then layer on a “meta-selector” that chooses, per part, which baseline (SARIMA, GLM, ML) to use based on validation performance.

---

8. Concrete next steps for BMAD-METHOD PRD

Items to capture in prd.md:

- Objective: forecast daily part usage and rank top-20 parts per day to reduce stockouts and optimize inventory.
- Inputs: daily part usage matrix, calendar features, optional production metadata.
- Outputs: per-part forecast distribution parameters and a ranked list of 20 parts for next day.
- V1 scope:
    - SARIMA per part (log(1+count) transformed).
    - Simple Poisson/negative-binomial GLMs.
    - Top-20 ranking using expected counts.
    - Rolling-origin backtesting and dashboard of key metrics.
- V2 scope:
    - Multivariate and ML forecasters.
    - Probabilistic ranking and inventory-level decision support (e.g., recommend order quantities).

Items to capture in architecture.md:

- Data flow diagram: raw CSV → cleaned time series → feature store → model training → forecast store → ranking output.
- Components and services, their APIs, and update cadence.
- Model lifecycle management (versioning, retraining policies, monitoring).

If you want, I can next:

- Propose a concrete SARIMA + GLM API (function signatures, data schemas).
- Or draft skeleton prd.md and architecture.md text tailored to BMAD-METHOD conventions.

Prepared using Nemotron 3 Super

I forgot to strip-out the final col w/ '30'. I was just running a quit data check to make sure the totals each day were correct. I have now removed the last col but it is true 30 total parts are used/shipped each day. Make changes as necessary and provide a new concept doc

Completed 1 step

The revised constraint "exactly 30 total parts per day across P_1…P_39" makes your data a *compositional, constrained multivariate count time series*, which changes model choice and interpretation but SARIMA can still play a role as a baseline per-part forecaster within a richer framework.

Below is an updated concept document reflecting that.

Updated problem framing

You now have:

- One row per **date** and 39 columns **P_1…P_39**, each a non-negative integer count of parts used/shipped that day.
- The **row sum is always 30**: $\sum_{j=1}^{39} Y_t^{(j)} = 30$ for all days t.

Interpretation:

- Each day is a 39-category count vector of total size 30, i.e. a multinomial outcome per day over parts 1–39.
- Marginally, each part's series $\left\{Y_t^{(j)}\right\}$ is still a daily count time series, but they are *strongly dependent* through the fixed-total constraint.

Prediction goal (unchanged in spirit, sharpened in math):

- For a future day $t+1$, predict the distribution over the 39-dimensional count vector $Y_{t+1} = (Y_{t+1}^{(1)}, \dots, Y_{t+1}^{(39)})$ subject to $\sum_j Y_{t+1}^{(j)} = 30$, and from that obtain the 20 parts with highest probability of positive usage (or largest expected counts).

Implications of the "30 per day" constraint

Key structural consequences:

- **Negative dependence**: if one part's count is high on a given day, some other parts must be lower so that the total is 30.
- **Compositional data**: the relative proportions $p_{t,j} = Y_t^{(j)}/30$ form a composition in a 39-part simplex for each day t.
- **Multinomial generative view**: a natural first approximation is $Y_t \sim \text{Multinomial}(n = 30, \pi_t)$ with category probability vector π_t that evolves over time.

This argues for **multivariate or compositional time-series models** in addition to per-part univariate methods.

Role of SARIMA in the new setting

SARIMA is still viable as:

- A **per-part marginal forecaster** for $Y_t^{(j)}$ or a transformation thereof.
- A baseline to benchmark more structural multinomial/compositional models.

However:

- A SARIMA for each part, run independently, will generally **not respect** the 30-total constraint on the joint vector $\left(\hat{Y}_{t+1}^{(1)}, \dots, \hat{Y}_{t+1}^{(39)}\right)$.
- You will need a **normalization or post-processing** step if you want forecasts consistent with the known total.

Two main ways to adapt SARIMA:

1. Model each part's *proportion* series $p_{t,j} = Y_t^{(j)}/30$ with SARIMA (after logit/log-ratio transforms), then renormalize to sum to 1 when forming $\hat{\pi}_{t+1}$.
2. Use SARIMA to forecast marginal counts, then rescale the 39 predictions so that they sum to 30 while preserving rank/order (e.g., multiply by a common factor).

This makes SARIMA part of a **marginal-forecast + joint-constraint reconciliation** pipeline.

Recommended updated modeling pipeline

1. Data preprocessing

- Keep only columns date, P_1, ..., P_39; assert that each row sums to 30 as a data quality check.
- Create *proportion* features: $p_{t,j} = Y_t^{(j)}/30$, so each row of $P_t = (p_{t,1}, \dots, p_{t,39})$ lies on the simplex and encodes composition.
- Add calendar and exogenous features as before (day-of-week, month, holidays, etc.).

Deliverable: updated data_prep.py and schema that explicitly documents the fixed-total property.

---

2. Compositional / multinomial modeling (structural view)

Given the multinomial interpretation, define:

- $Y_t \mid \pi_t \sim \text{Multinomial}(n = 30, \pi_t)$, where π_t is a 39-dimensional probability vector for day t.
- The time series problem is to model and forecast π_t.

Two conceptually strong families:

2.1 Log-ratio transformed time series

- Choose a reference part, e.g., part 39, and form additive log-ratio (ALR) transforms:

$$z_{t,j} = \log\left(\frac{p_{t,j}}{p_{t,39}}\right), j = 1, \dots, 38$$

yielding an unconstrained real-valued 38-dimensional time series.

- For each coordinate $z_{t,j}$, fit SARIMA/SARIMAX or other univariate models; optionally, use vector models (VAR) if you want cross-coordinate dynamics.
- For prediction:
    - Forecast $\hat{z}_{t+1,j}$.
    - Back-transform to proportions:

$$\tilde{p}_{t+1,j} = \exp\left(\hat{z}_{t+1,j}\right), \tilde{p}_{t+1,39} = 1, \hat{p}_{t+1,j} = \frac{\tilde{p}_{t+1,j}}{\sum_{k=1}^{39} \tilde{p}_{t+1,k}}$$

- Set $\hat{Y}_{t+1}^{(j)} = 30 \cdot \hat{p}_{t+1,j}$, optionally rounded.

This approach:

- Uses SARIMA in a way that naturally respects the 30-total constraint because proportions are normalized on back-transform.
- Gives a **probability vector** $\hat{\pi}_{t+1}$ directly for downstream multinomial modeling and ranking.

2.2 Direct multinomial logistic modeling

- Model π_t via a multinomial logistic regression or softmax from latent scores $s_{t,j}$:

$$\pi_{t,j} = \frac{\exp\,(s_{t,j})}{\sum_k \exp\,(s_{t,k})}$$

- The scores $s_{t,j}$ can be parameterized by:
    - Linear models with lags of Y or p, calendar features, etc.
    - Recurrent or transformer-style time-series models producing a 39-dimensional score per day.

This is more of a machine-learning route, but conceptually aligned with the multinomial structure; SARIMA then becomes a baseline or a sub-component for generating features (e.g., trend/seasonality adjustments).

---

3. SARIMA as a compositional baseline

Under the ALR approach above, SARIMA's role becomes clearer:

- For each transformed coordinate $z_{t,j}$, fit a SARIMA/SARIMAX model capturing autocorrelation and seasonality.
- Model selection, diagnostics, and periodicity (e.g., weekly) proceed as before, but on continuous, approximately Gaussian series.
- Forecasted $\hat{z}_{t+1,j}$ are then back-transformed into a valid simplex vector $\hat{p}_{t+1}$ and counts $\hat{Y}_{t+1}$.

Advantages:

- Respects the "30 total" constraint by construction.

- Gives interpretable marginal trajectories on log-ratios (e.g., how part j's share evolves relative to reference part 39).

Caveats:

- The choice of reference part can influence interpretability; you may prefer centered log-ratio (CLR) transforms, though they require handling the sum-to-zero constraint in modeling.

Deliverable: models/compositional_sarima.py implementing ALR transforms, SARIMA fitting, and back-transformation.

---

4. Ranking logic under fixed total

Given a forecasted composition $\hat{p}_{t+1}$ and therefore expected counts $\hat{Y}_{t+1}^{(j)} = 30 \cdot \hat{p}_{t+1,j}$:

- The **top-20 parts** are simply those with the 20 largest $\hat{p}_{t+1,j}$ or $\hat{Y}_{t+1}^{(j)}$.
- Because the total is fixed at 30, ranking by expected counts or probabilities is equivalent (monotone transform).

For probabilistic ranking (beyond expectations):

- If you adopt the multinomial view, you can approximate:
    - $\Pr\,(Y_{t+1}^{(j)} \geq 1) = 1 - (1 - \hat{p}_{t+1,j})^{30}$.
- This gives a probability that each part appears at least once in the next day's 30 draws, which can be another ranking metric.

Evaluation metrics for the ranking (unchanged structurally):

- Precision@20, Recall@20 using event "$Y_{t+1}^{(j)} > 0$".
- nDCG with true counts as relevance, still computed on top-20 items.

Deliverable: updated ranking/top_k_selector.py that operates on predicted compositions and exposes both expected counts and probabilities of non-zero.

---

5. Alternative / complementary models

Beyond compositional SARIMA, consider:

5.1 Multivariate time-series on log-ratios

- Vector autoregression (VAR) on z_t (ALR or CLR) to explicitly capture cross-part temporal interactions within the log-ratio space.
- Dynamic factor models on z_t to learn latent demand patterns across parts, then map back to compositions.

These naturally incorporate the fixed-sum structure after back-transformation and can outperform independent SARIMAs when parts are strongly coupled.

5.2 Neural / ML sequence models on multinomial outputs

- Sequence-to-one or sequence-to-sequence models that output 39 logits per day, normalized with softmax to a probability vector.
- Training objective: minimize negative log-likelihood under a multinomial (or equivalently, categorical cross-entropy over the 30 "tokens" with counts as weights).

These preserve the sum-to-30 structure directly and can exploit rich features; SARIMA then acts as a transparent baseline and an interpretable component.

---

6. Evaluation and comparison in this setting

Per-part forecast evaluation:

- Use per-part MAE/RMSE on counts as before, but also check **simplex-level** metrics such as:
    - Kullback–Leibler (KL) divergence between true composition p_t and predicted $\hat{p}_t$.
    - Aitchison distance (appropriate for compositional data).

Top-20 ranking evaluation:

- Same rank-based metrics (Precision@K, Recall@K, nDCG) but emphasize that *only 30 of 39 parts can be non-zero each day*, so the upper bound of recall is tightly bound.

Backtesting protocol:

- Rolling-origin cross-validation as before, but with checks that the predicted compositions always sum to 1 and implied counts sum to 30 (model sanity).

Deliverable: evaluation/composition_metrics.py with KL, Aitchison, and existing ranking metrics.

---

7. Architecture / PRD updates

For prd.md:

- Explicitly state: "Each day consists of 30 units allocated across 39 part categories; the system must produce forecasts that respect this structural constraint, or explicitly document and correct any violations."
- Include compositional-data concepts in the "Modeling Approach" section: ALR/CLR transforms, multinomial interpretation, and why this matters.
- Define V1 scope as:
    - Compositional SARIMA baseline via ALR + SARIMA per coordinate.
    - Basic multinomial-derived probability-of-use and top-20 ranking.

For architecture.md:

- In the modeling service description, add a **"composition transformer"** step:
    - Convert raw counts → compositions → log-ratios → models → predicted log-ratios → compositions → counts.
- Ensure the model monitoring section includes checks that:
    - Predicted compositions are valid probability simplices (non-negative, sum to 1).
    - Implied counts sum to 30; if you layer additional models that don't naturally enforce this, add a reconciliation module.

---

8. Concrete deltas vs previous concept

Relative to the first concept doc, the main changes you would highlight to the BMAD-METHOD and Claude Code teams:

- Treat daily vectors as **multinomial/compositional**, not independent Poisson-like counts.

- Use **log-ratio transforms + SARIMA** as the principled way to keep the “30 total” constraint while still leveraging SARIMA.
- Evaluate both **per-part accuracy and composition-level divergences**.
- Keep the ranking logic, evaluation metrics, and broader architecture largely as before, but insert explicit composition handling and constraint checks.

If you’d like, I can now draft concrete pseudocode / interface stubs for:

- fit_compositional_sarima(df) and predict_next_day_composition(model, history).